Project: House Price Prediction

Dataset Used: kc_house_data.csv

This document outlines a structured, complete report of project based on training the module, predicting the test data and evaluating the accuracy on given test data.

Project Goal: Train a module to predict the classes using kc_house_data.csv dataset.

The primary libraries/modules required for this project are:

- **Pandas**: For handling datasets.
- **NumPy**: For numerical operations.
- **sklearn**: For statistical machine learning.

Project Coverage:

Category	Skills Covered
Data Pre-Processing	Data Loading.
Create Learning Dataset	Select the required columns for passing as Input.
Training the module	Algorithms to train the module using the dataset.
Predict the result	Algorithms to predict the species class

In the code we are importing the required libraries to load the dataset and creating a training set to train the module. After training using the dataset, we are using different classes from the sklearn module to predict the species class with different algorithms as shown below.

1. Multiple Linear Regression

Linear Regression models the relationship between features and price by fitting a linear equation (a straight line) to the data.

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
print(regressor)

regressor.fit(X_train, y_train)
y_pred = regressor.predict(X_test)

print('\n'+'-'*20+'R2 Score on the Test set'+'-'*20)
print("{:.2f}".format(r2_score(y_test, y_pred)))
```

In this algorithm we are importing LinearRegression class from linear_model sub-module and using fit function to train the module. After that we are predicting the accuracy of that module and printing it.

2. Decision Tree Regression

This model breaks down the dataset into smaller subsets while simultaneously developing an associated decision tree (flowchart-like).

```
from sklearn.tree import DecisionTreeRegressor
regressor = DecisionTreeRegressor(random_state=0)
print(regressor)

regressor.fit(X_train, y_train)
y_pred = regressor.predict(X_test)

print('\n'+'-'*20+'R2 Score on the Test set'+'-'*20)
print("{:.2f}".format(r2_score(y_test, y_pred)))
```

In this algorithm we are importing DecisionTreeRegressor class from tree sub-module and using fit function to train the module. After that we are predicting the accuracy of that module and printing it.

3. Random Forest Regression

An ensemble method that operates by constructing many decision trees and outputting the average prediction of the individual trees.

```
from sklearn.ensemble import RandomForestRegressor
regressor = RandomForestRegressor(n_estimators=100, random_state=0)
print(regressor)

regressor.fit(X_train, y_train)
y_pred = regressor.predict(X_test)

print('\n'+'-'*20+'R2 Score on the Test set'+'-'*20)
print("{:.2f}".format(r2_score(y_test, y_pred)))
```

In this algorithm we are importing RandomForestRegressor class from ensemble sub-module and using fit function to train the module. After that we are predicting the accuracy of that module and printing it.

4. Support Vector Regression (SVR)

SVR tries to fit the error within a certain threshold (epsilon) instead of just minimizing the error, using kernels for non-linear data.

```
from sklearn.svm import SVR
regressor = SVR(kernel='rbf')
print(regressor)

regressor.fit(X_train, y_train)
y_pred = regressor.predict(X_test)

print('\n'+'-'*20+'R2 Score on the Test set'+'-'*20)
print("{:.2f}".format(r2_score(y_test, y_pred)))
```

In this algorithm we are importing SVR class from svm sub-module and using fit function to train the module. After that we are predicting the accuracy of that module and printing it.

5. K-Nearest Neighbors (KNN)

KNN predicts the value of a data point based on how similar it is to its surrounding neighbors in the training set.

```
from sklearn.neighbors import KNeighborsRegressor
regressor = KNeighborsRegressor(n_neighbors=5)
print(regressor)

regressor.fit(X_train, y_train)
y_pred = regressor.predict(X_test)

print('\n'+'-'*20+'R2 Score on the Test set'+'-'*20)
print("{:.2f}".format(r2_score(y_test, y_pred)))
```

In this algorithm we are importing KNeighborsRegressor class from neighbors sub-module and using fit function to train the module. After that we are predicting the accuracy of that module and printing it.

6. Gradient Boosting Regression

This builds the model in stages; it adds new trees that focus specifically on correcting the errors made by previous trees.

```
from sklearn.ensemble import GradientBoostingRegressor
regressor = GradientBoostingRegressor(n_estimators=100, random_state=0)
print(regressor)

regressor.fit(X_train, y_train)
y_pred = regressor.predict(X_test)

print('\n'+'-'*20+'R2 Score on the Test set'+'-'*20)
print("{:.2f}".format(r2_score(y_test, y_pred)))
```

In this algorithm we are importing GradientBoostingRegressor class from ensemble sub-module and using fit function to train the module. After that we are predicting the accuracy of that module and printing it.

7. Ridge Regression

A type of linear regression that uses "L2 Regularization" to prevent overfitting by penalizing large coefficients.

```
from sklearn.linear_model import Ridge
regressor = Ridge(alpha=1.0)
print(regressor)

regressor.fit(X_train, y_train)
y_pred = regressor.predict(X_test)

print('\n'+'-'*20+'R2 Score on the Test set'+'-'*20)
print("{:.2f}".format(r2_score(y_test, y_pred)))
```

In this algorithm we are importing Ridge class from linear_model sub-module and using fit function to train the module. After that we are predicting the accuracy of that module and printing it.

Conclusion:

In this project from the given dataset we can trained different modules with different algorithms to predict price of the house. We can also see the accuracy of the algorithms that are used to train the modules. Once we pass the required values for the input the code will predict the price of the house and provide us with the price with more accuracy.

What you have learned:

In this project we learnt that to predict the price of house we can use different algorithms to train different modules and predict the price with more accuracy. And we can use the most suitable algorithm depending on the requirement.